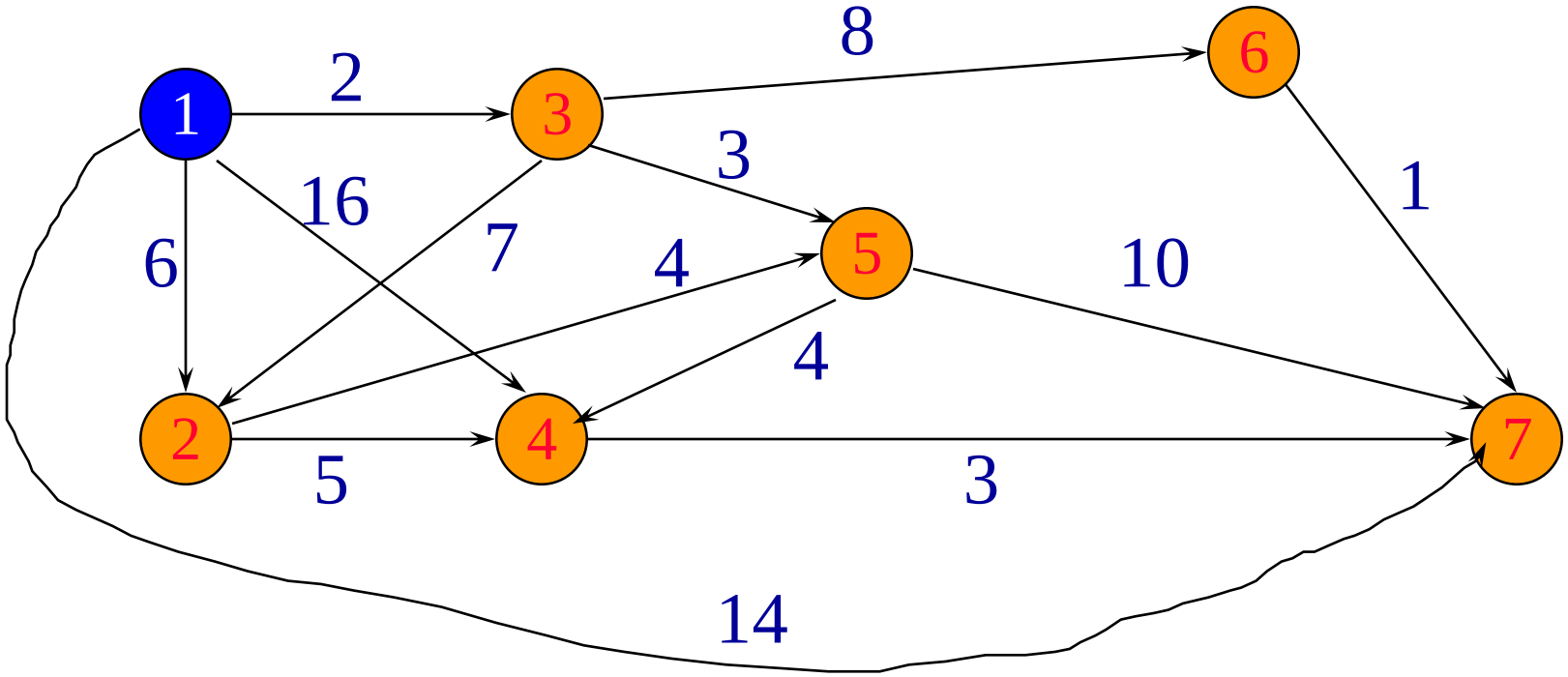
# Fibonacci Heaps

| | Actual | Amortized |
|---|---|---|
| Insert | O(1) | O(1) |
| Remove min (or max) | O(n) | O(log n) |
| Meld | O(1) | O(1) |
| Remove | O(n) | O(log n) |
| Decrease key (or increase) | O(n) | O(1) |

# Single Source All Destinations Shortest Paths



# Greedy Single Source All Destinations

- Known as Dijkstra's algorithm.
- Let d(i) be the length of a shortest one edge extension of an already generated shortest path, the one edge extension ends at vertex i.
- The next shortest path is to an as yet unreached vertex for which the d() value is least.
- After the next shortest path is generated, some d() values are updated (decreased).

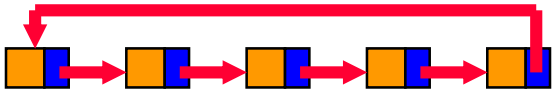
# Operations On d()

- Remove min.
  - Done O(n) times, where n is the number of vertices in the graph.
- Decrease d().
  - Done O(e) times, where e is the number of edges in the graph.
- Array.
  - $O(n^2)$ overall complexity.
- Min heap.
  - O(nlog n + elog n) overall complexity.
- Fibonacci heap.
  - O(nlog n + e) overall complexity.

# Min Fibonacci Heap

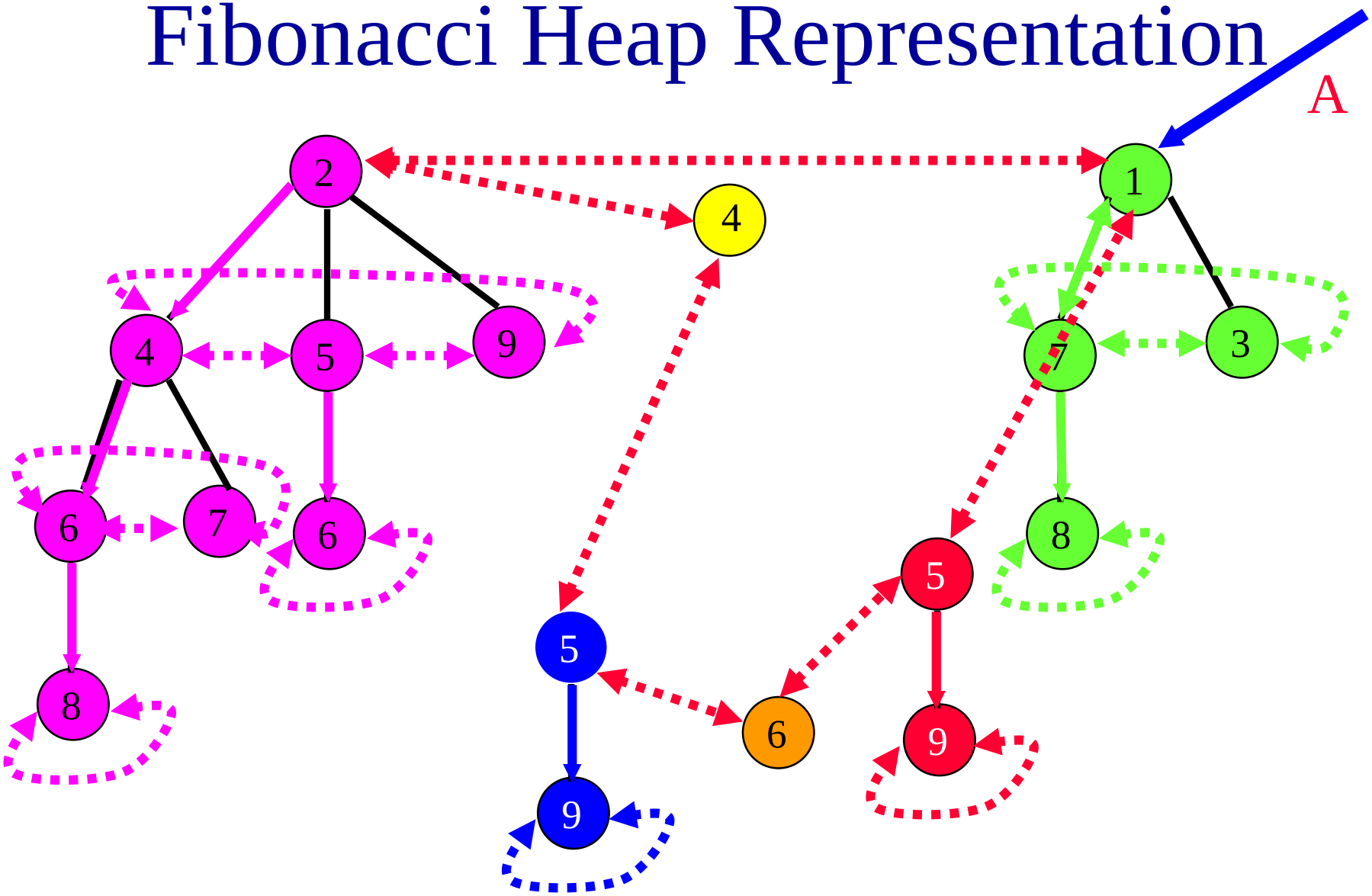
- Collection of min trees.

# Node Structure



- Degree, Child, Data
- Left and Right Sibling
  - Used for circular doubly linked list of siblings.
- Parent
  - Pointer to parent node.
- ChildCut
  - True if node has lost a child since it became a child of its current parent.
  - Set to false by remove min, which is the only operation that makes one node a child of another.
  - Undefined for a root node.

# Fibonacci Heap Representation

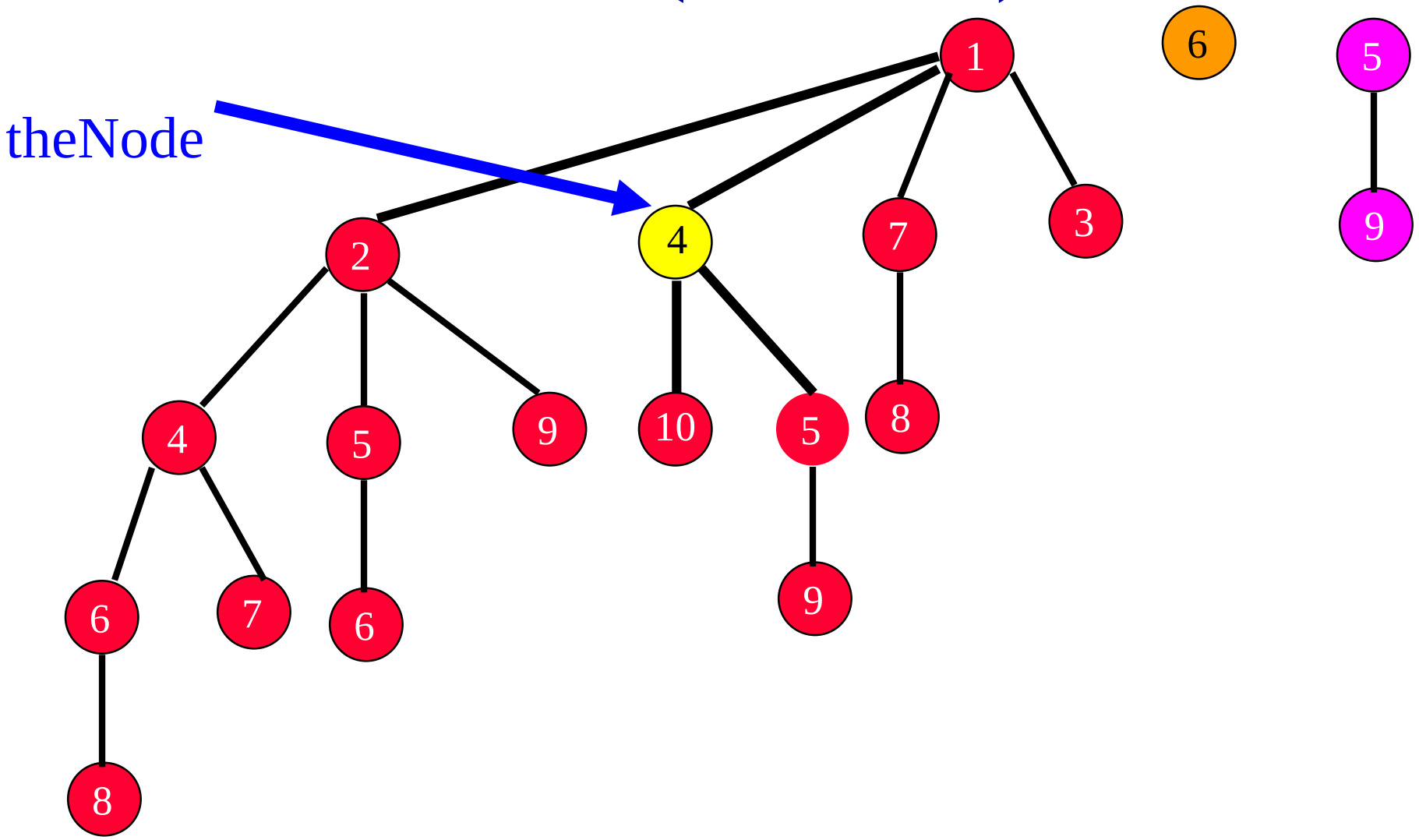



- Degree, Parent and ChildCut fields not shown.

# Remove(theNode)

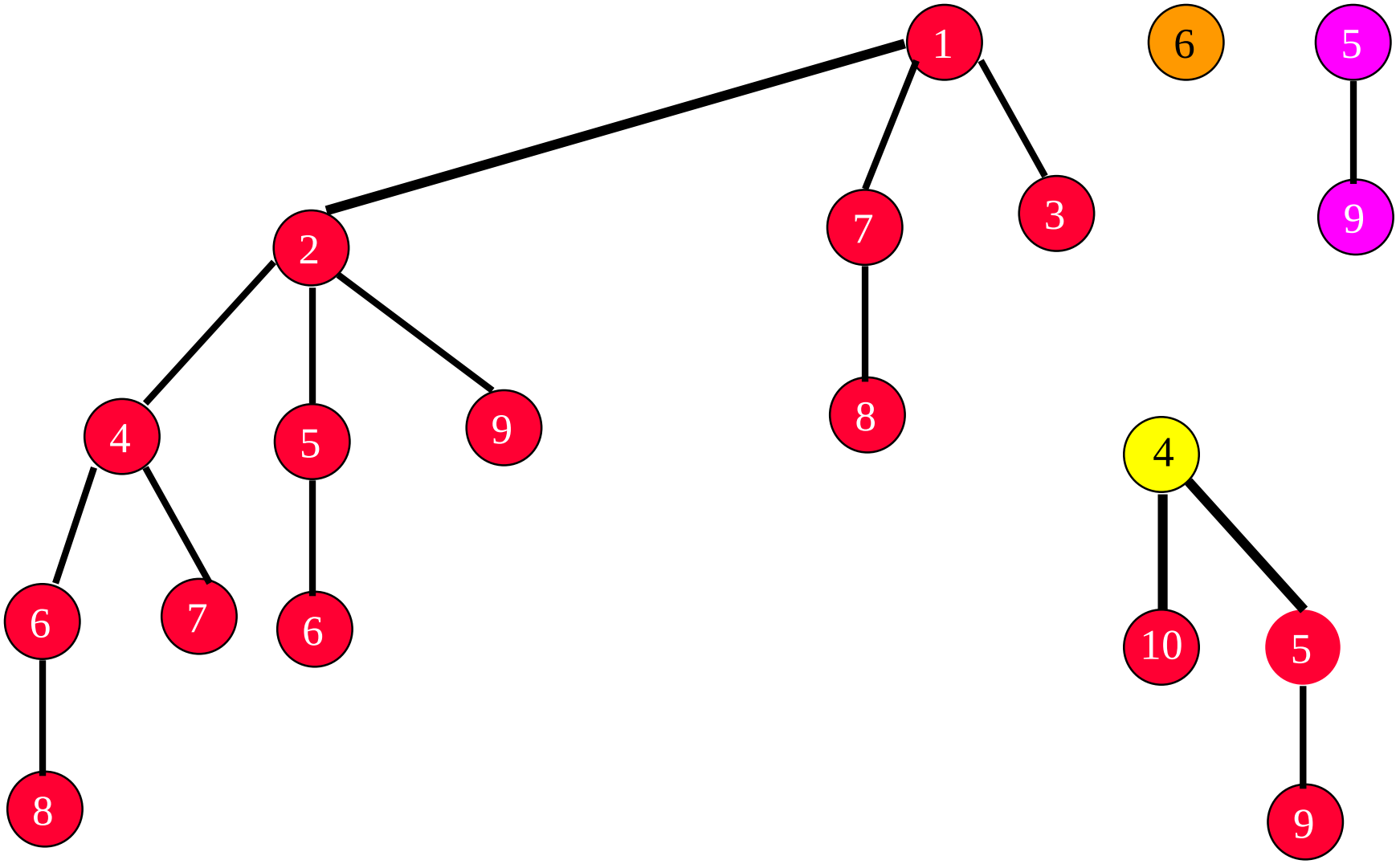
- theNode points to the Fibonacci heap node that contains the element that is to be removed.
- theNode points to min element => do a remove min.
  - In this case, complexity is the same as that for remove min.

# Remove(theNode)



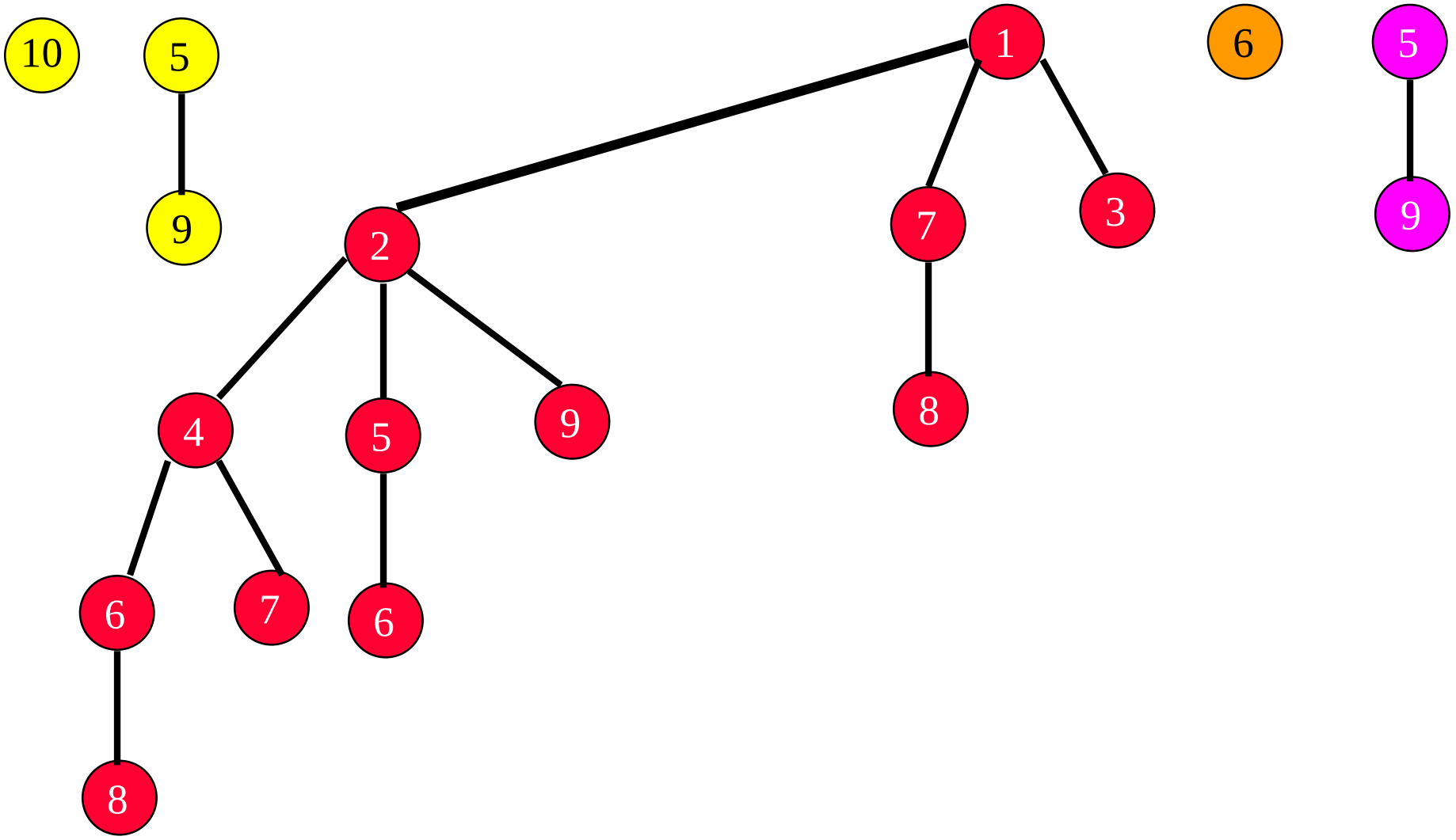

Remove theNode from its doubly linked sibling list.

# Remove(theNode)

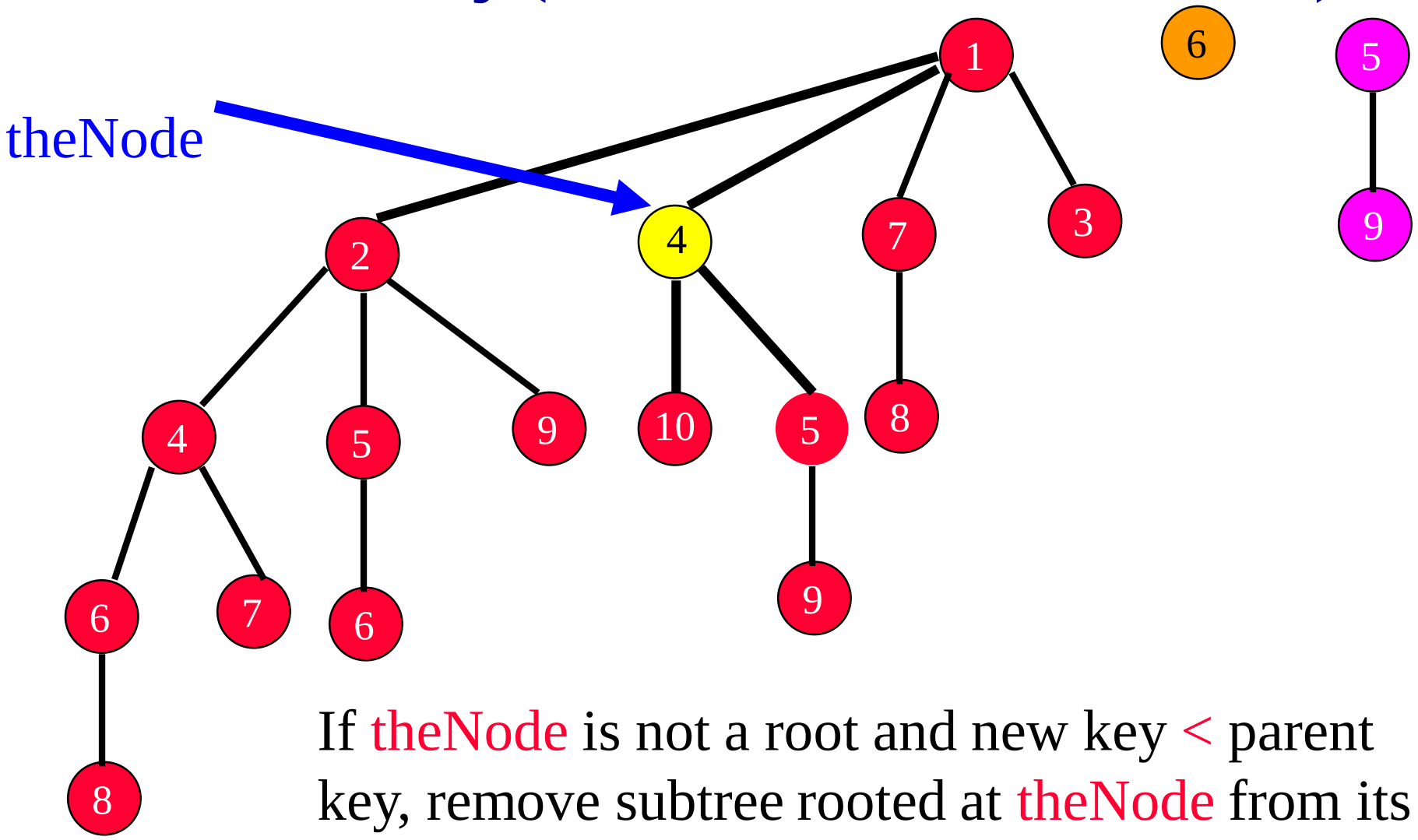



Combine top-level list and children of theNode setting parent pointers of the children of theNode to null.

# Remove(theNode)



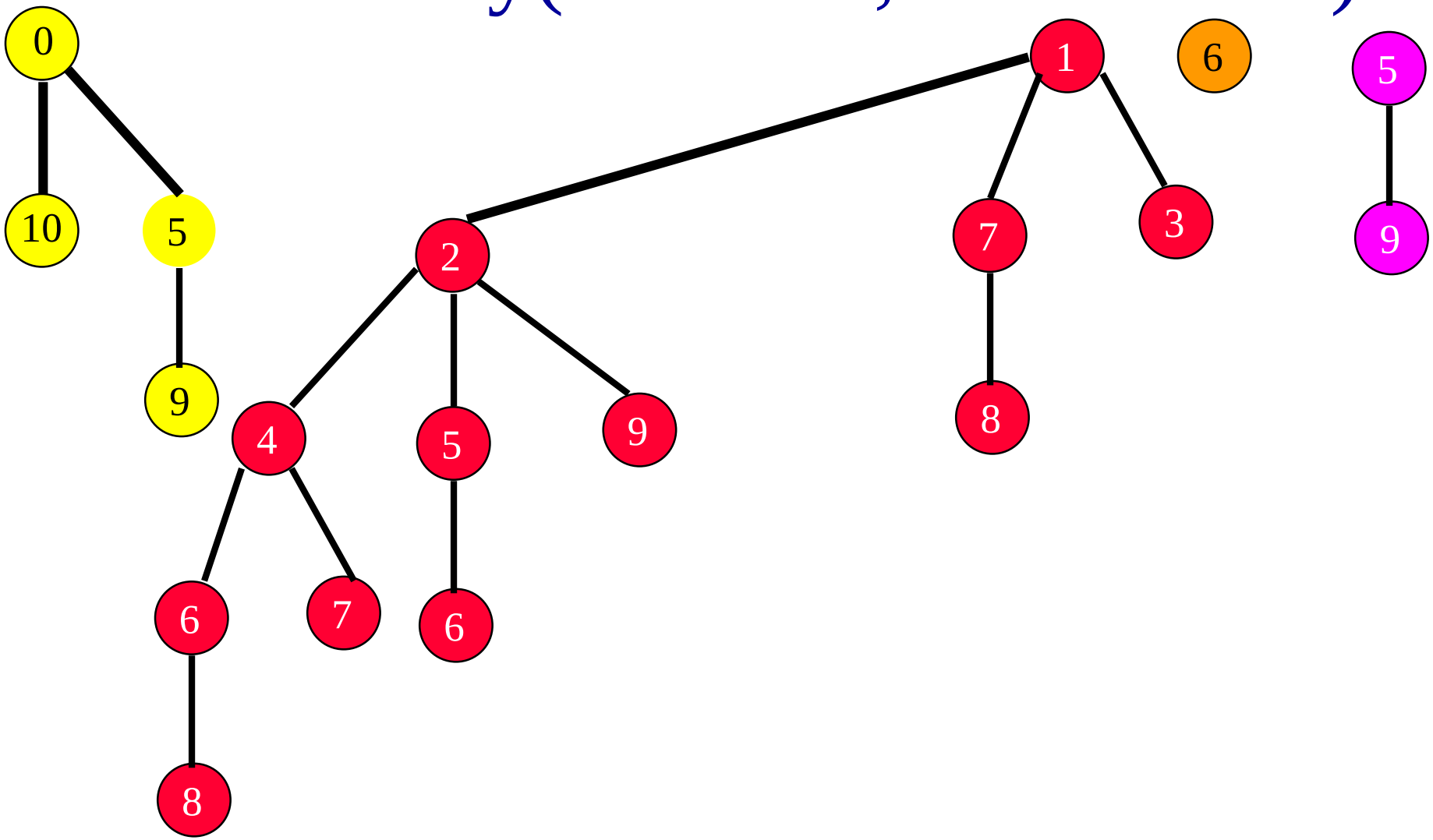
# DecreaseKey(theNode, theAmount)




If theNode is not a root and new key < parent key, remove subtree rooted at theNode from its doubly linked sibling list.

Insert into top-level list.

# DecreaseKey(theNode, theAmount)

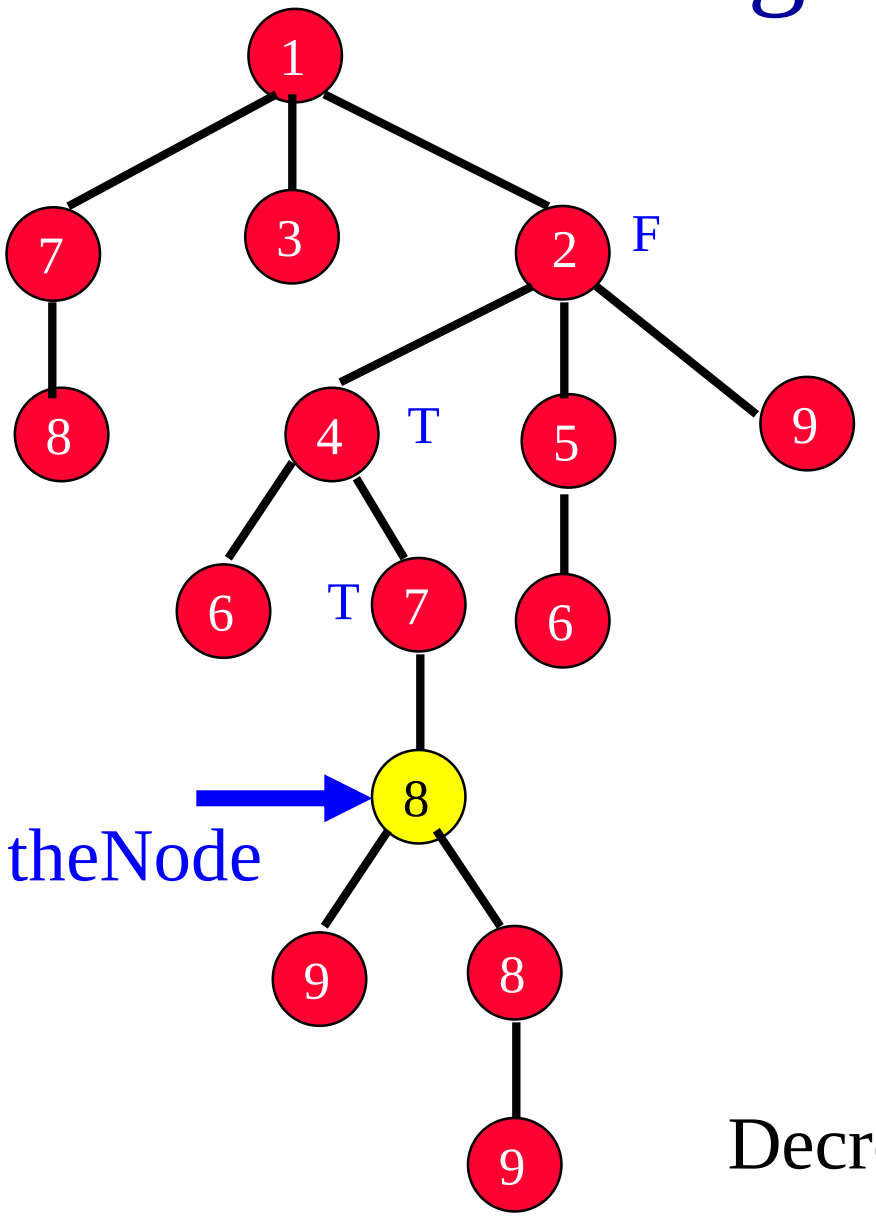



Update heap pointer if necessary

# Cascading Cut

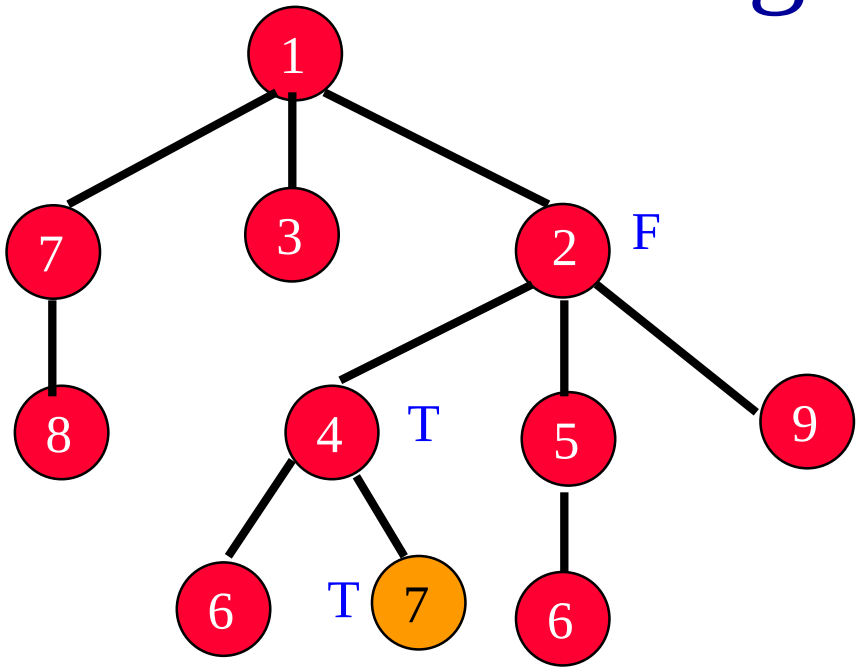
- When theNode is cut out of its sibling list in a remove or decrease key operation, follow path from parent of theNode to the root.
- Encountered nodes (other than root) with ChildCut = true are cut from their sibling lists and inserted into top-level list.
- Stop at first node with ChildCut = false.
- For this node, set ChildCut = true.

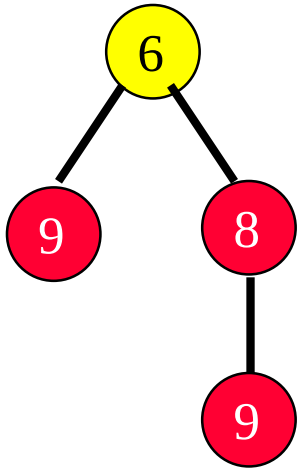
# Cascading Cut Example



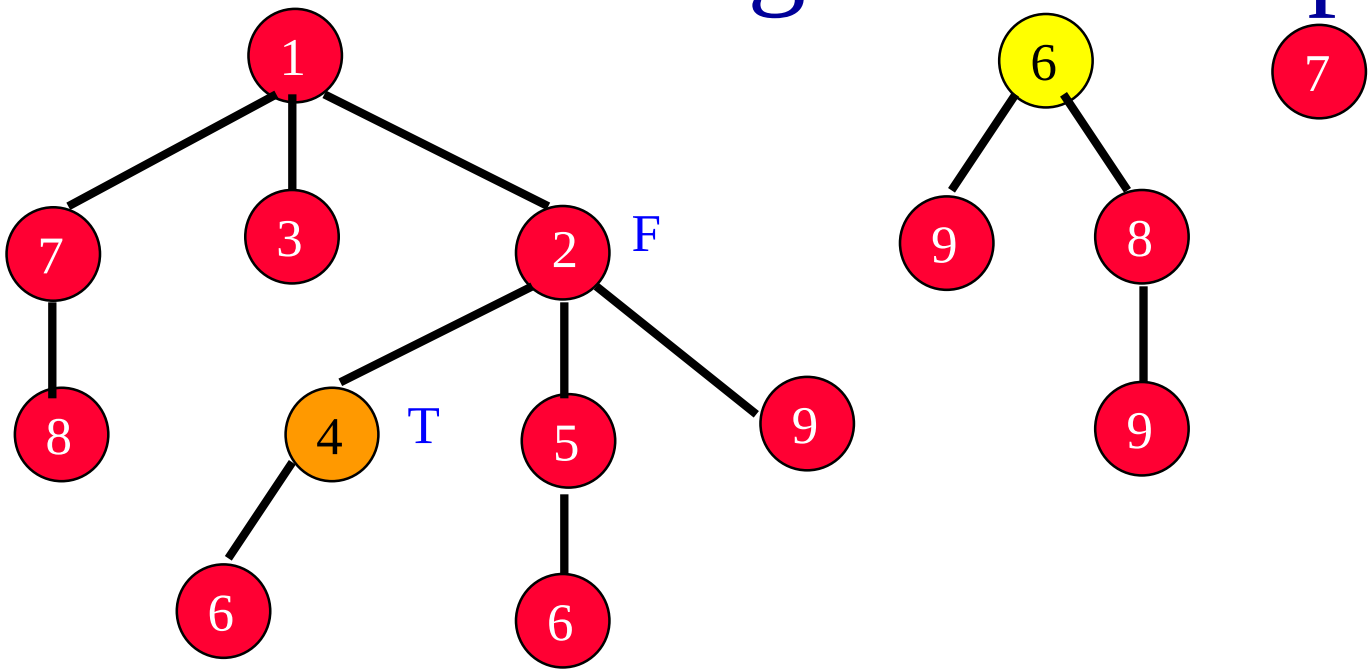

Decrease key by 2.

# Cascading Cut Example

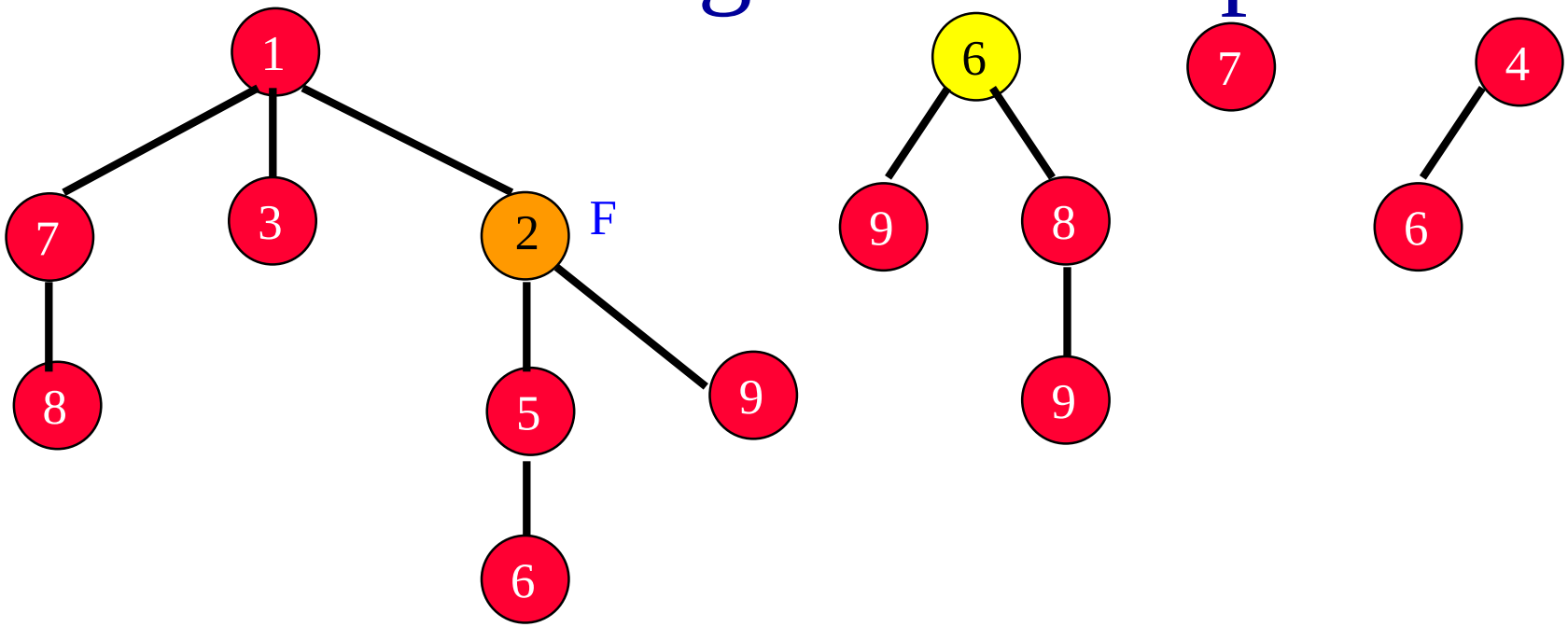


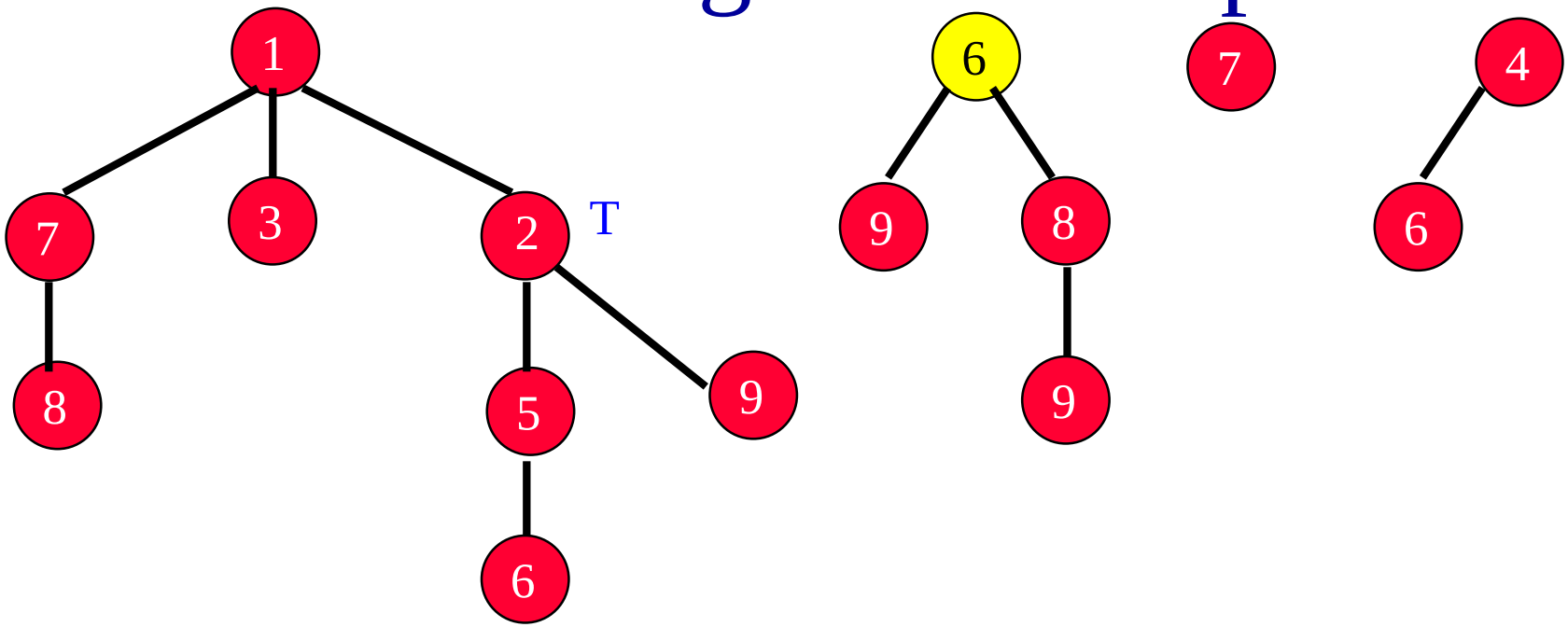
# Cascading Cut Example



# Cascading Cut Example



# Cascading Cut Example




Actual complexity of cascading cut is O(h) = O(n).

# Pairing Heaps

| | Fibonacci | Pairing |
|---|---|---|
| Insert | O(1) | O(1) |
| Remove min (or max) | O(n) | O(n) |
| Meld | O(1) | O(1) |
| Remove | O(n) | O(n) |
| Decrease key (or increase) | O(n) | O(1) |

**Actual Complexity**

# Pairing Heaps

| | Fibonacci | Pairing |
|---|---|---|
| Insert | O(1) | O(log n) |
| Remove min (or max) | O(log n) | O(log n) |
| Meld | O(1) | O(log n) |
| Remove | O(log n) | O(log n) |
| Decrease key (or increase) | O(1) | O(log n) |

**Amortized Complexity**

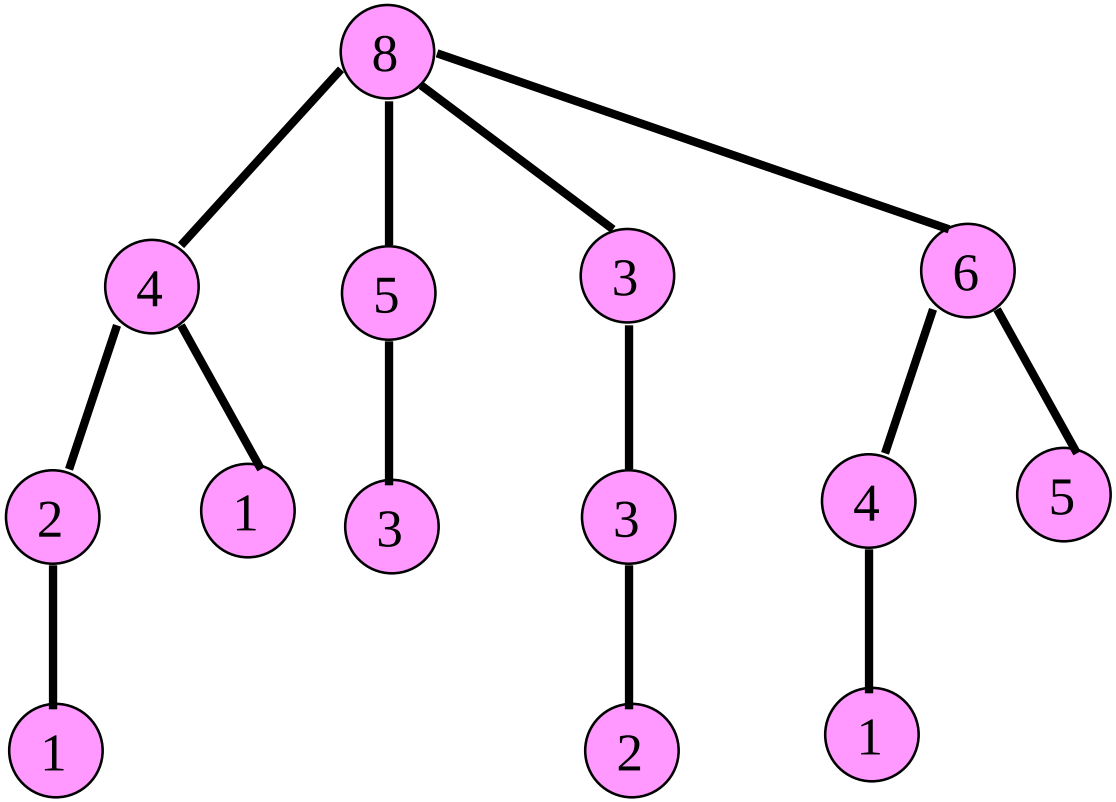
# Pairing Heaps

- Experimental results suggest that pairing heaps are actually faster than Fibonacci heaps.
  - Simpler to implement.
  - Smaller runtime overheads.
  - Less space per node.

# Definition

- A min (max) pairing heap is a min (max) tree in which operations are done in a specified manner.



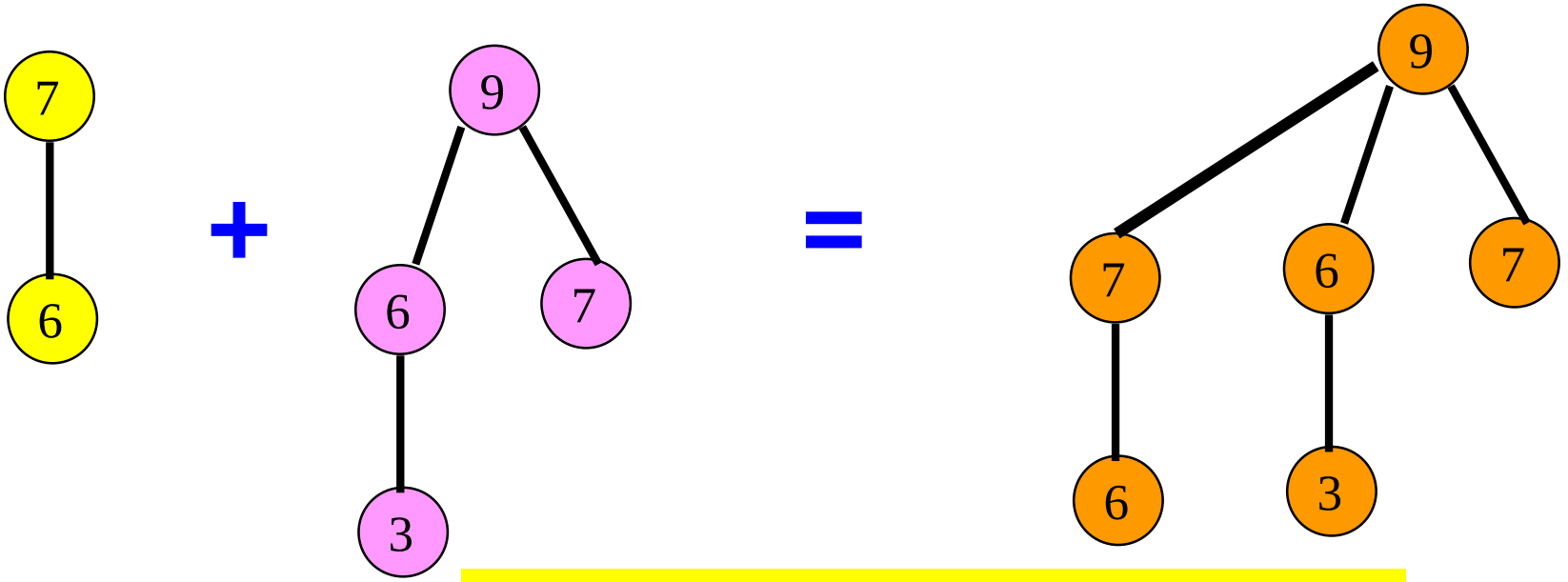



# Node Structure

- Child
  - Pointer to first node of children list.
- Left and Right Sibling
  - Used for doubly linked linked list (not circular) of siblings.
  - Left pointer of first node is to parent.
  - x is first node in list iff x.left.child = x.
- Data
- Note: No Parent, Degree, or ChildCut fields.

# Meld – Max Pairing Heap

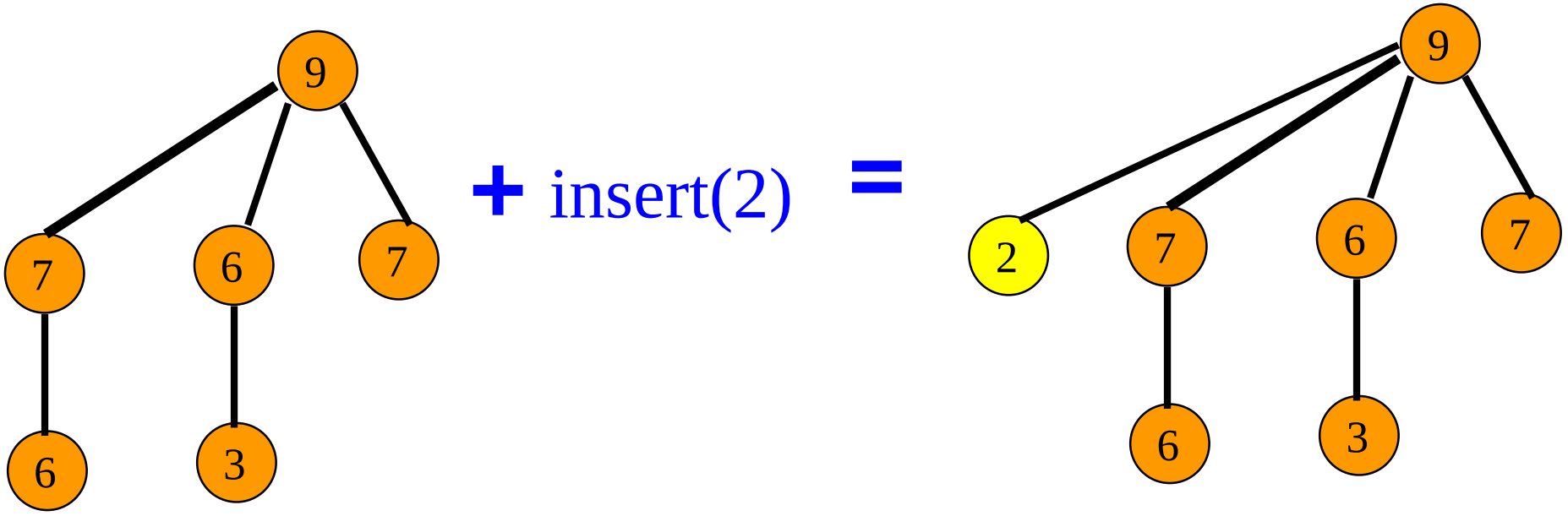
- Compare-Link Operation
  - Compare roots.
  - Tree with smaller root becomes leftmost subtree.




- Actual cost = O(1).

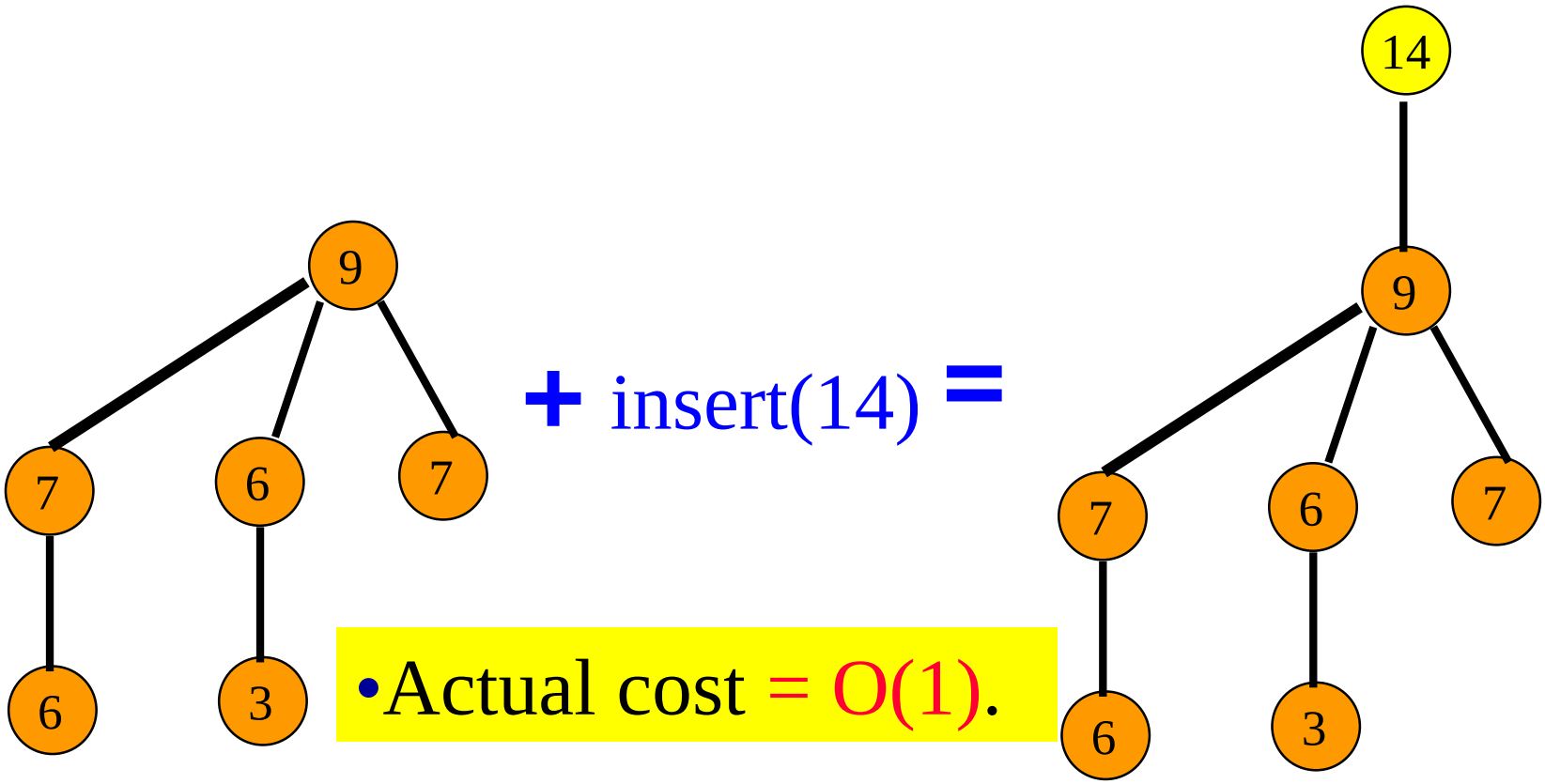
# Insert

- Create 1-element max tree with new item and meld with existing max pairing heap.



# Insert

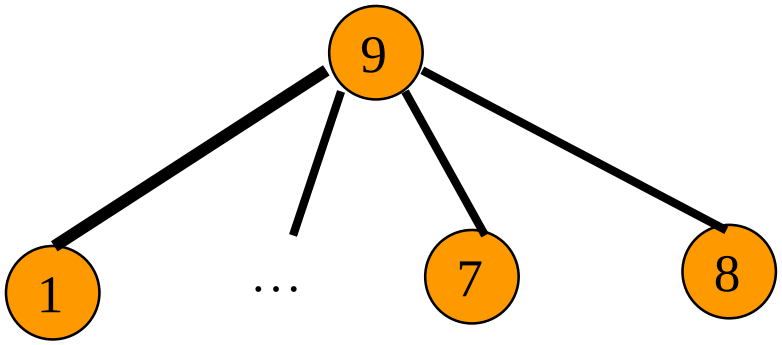
- Create 1-element max tree with new item and meld with existing max pairing heap.




- Actual cost = O(1).

# Worst-Case Degree

- Insert 9, 8, 7, …, 1, in this order.




- Worst-case degree = n –1.

# Worst-Case Height

- Insert 1, 2, 3, …, n, in this order.

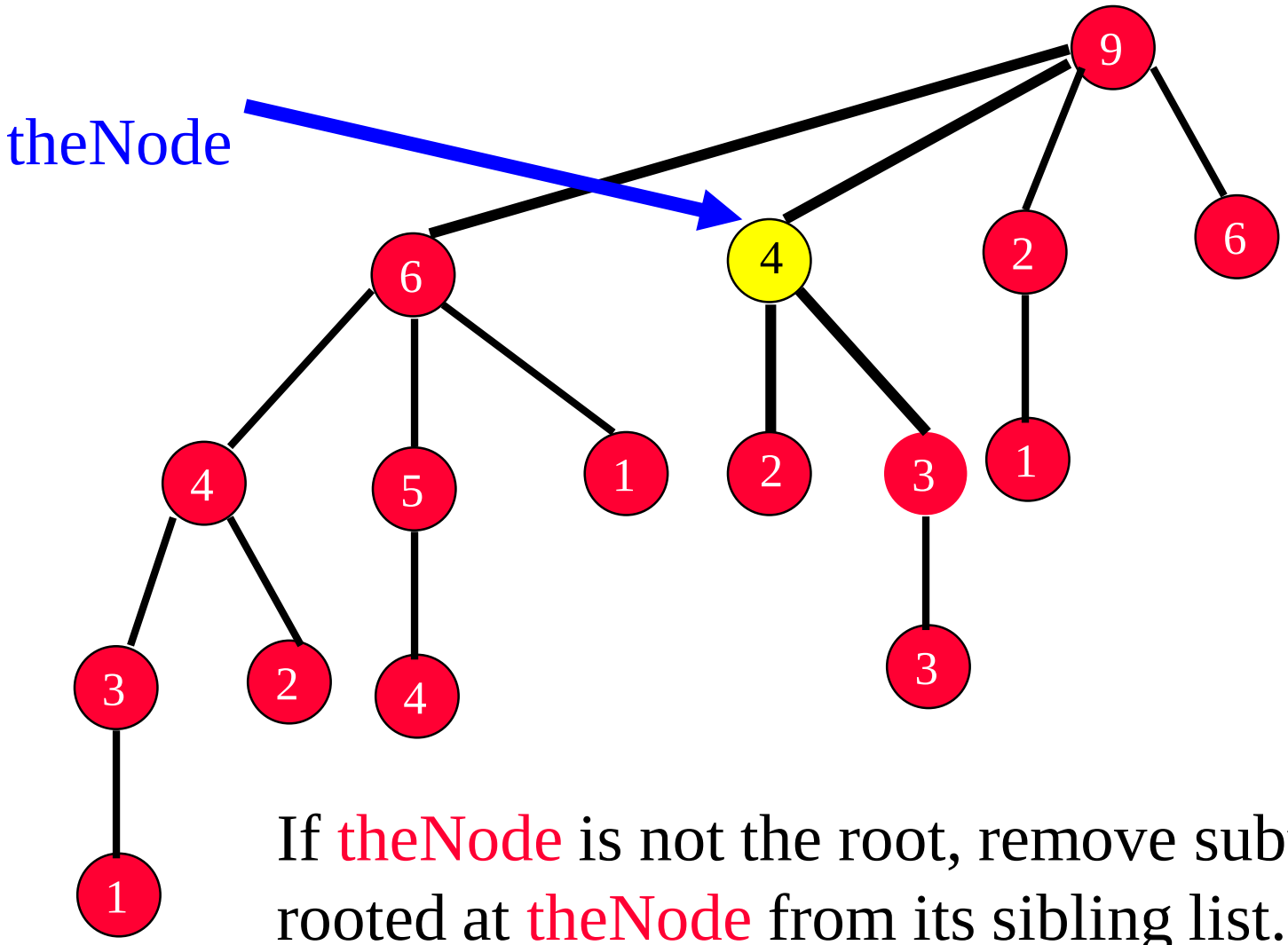



- Worst-case height = n.

# IncreaseKey(theNode, theAmount)

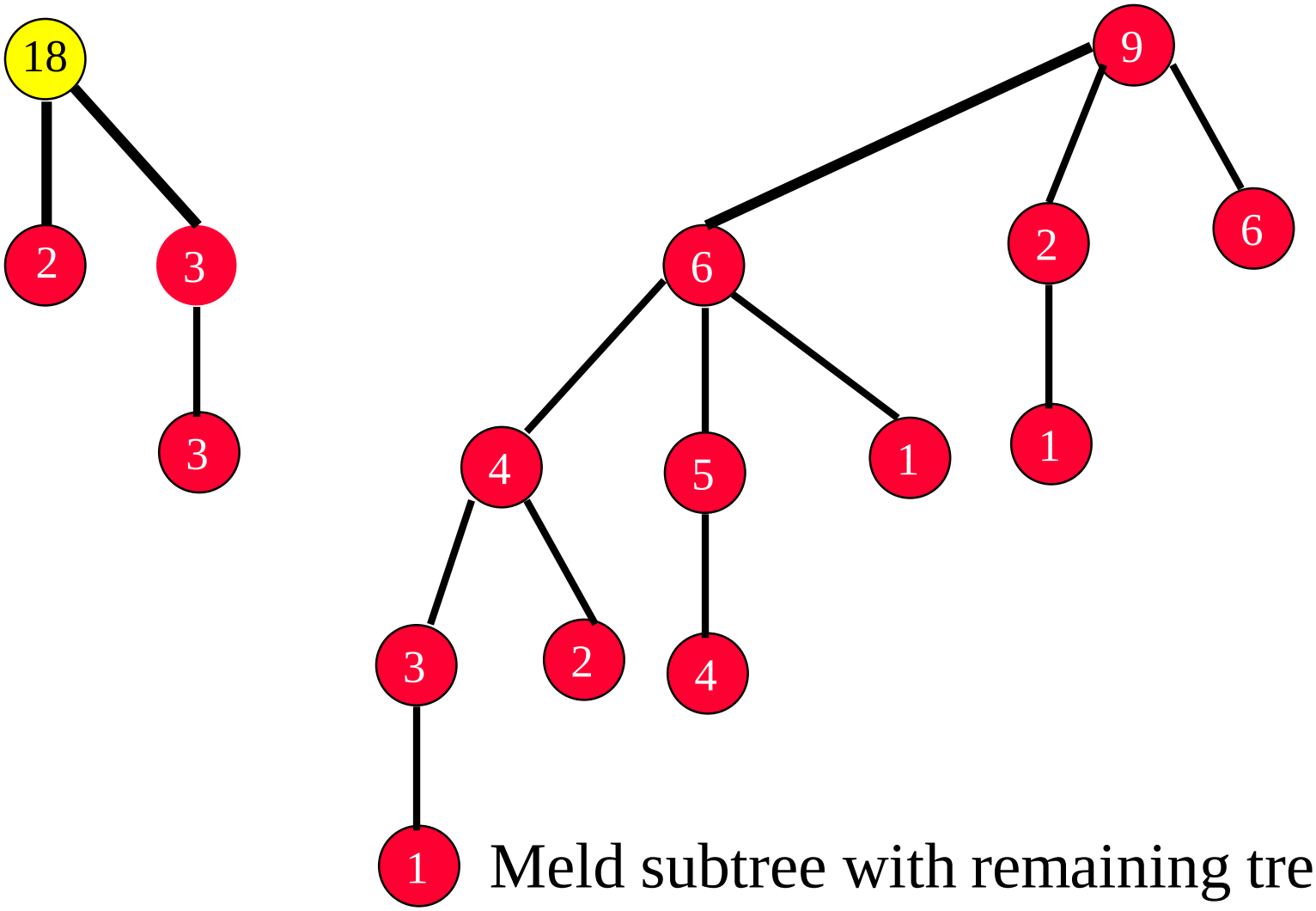
- Since nodes do not have parent fields, we cannot easily check whether the key in theNode becomes larger than that in its parent.
- So, detach theNode from sibling doubly-linked list and meld.

# IncreaseKey(theNode, theAmount)



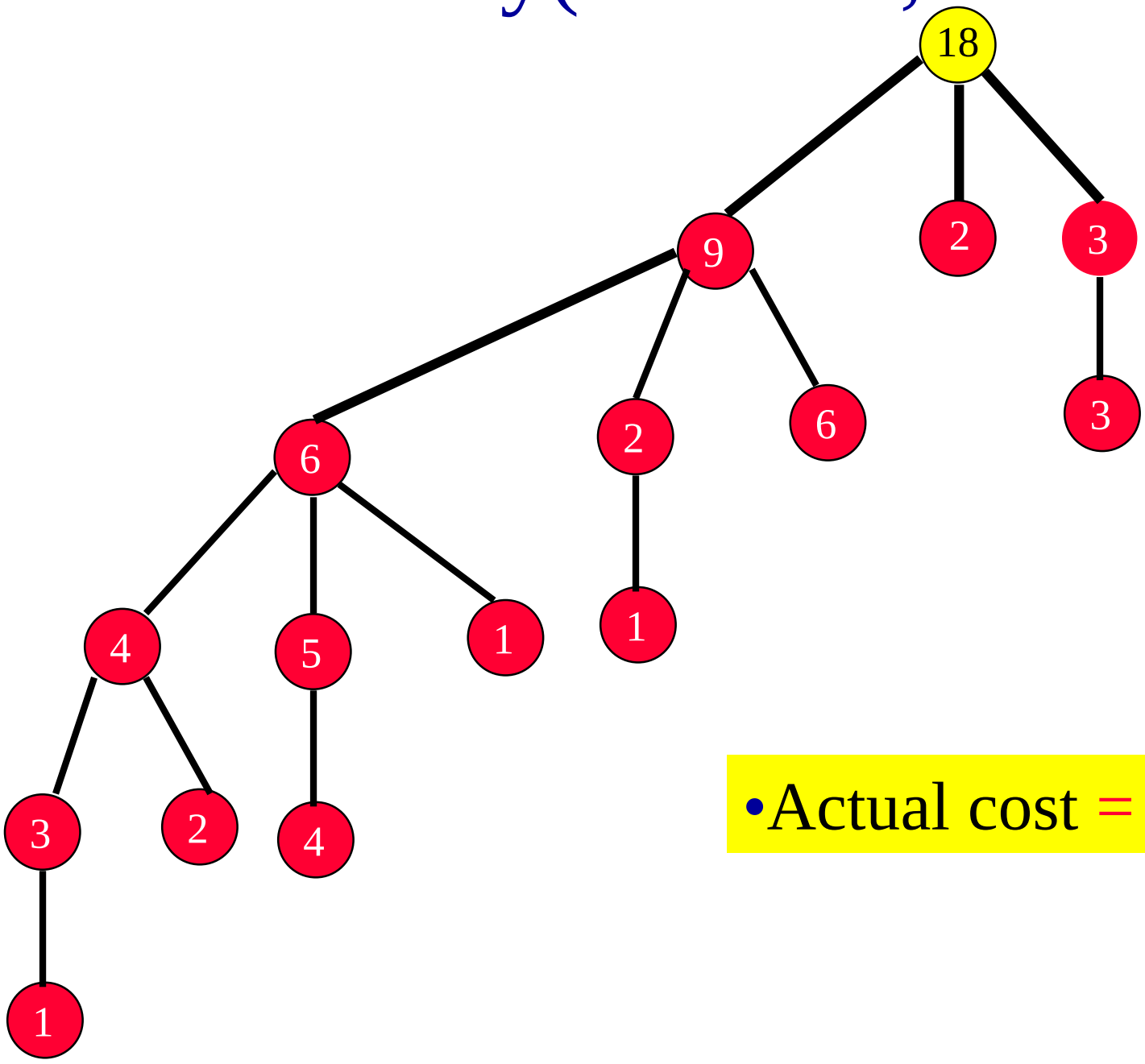

If theNode is not the root, remove subtree rooted at theNode from its sibling list.

# IncreaseKey(theNode, theAmount)




Meld subtree with remaining tree.

# IncreaseKey(theNode, theAmount)




- Actual cost = O(1).

# Remove Max

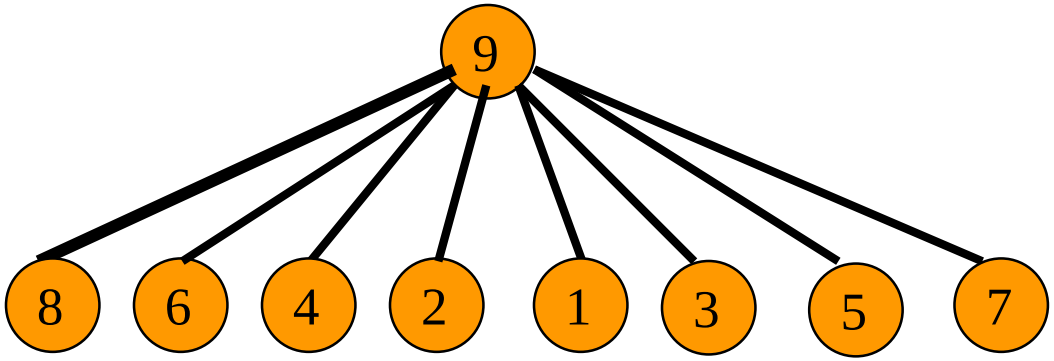
- If empty => fail.
- Otherwise, remove tree root and meld subtrees into a single max tree.
- How to meld subtrees?
  - Good way => O(log n) amortized complexity for remove max.
  - Bad way => O(n) amortized complexity.

# Bad Way To Meld Subtrees

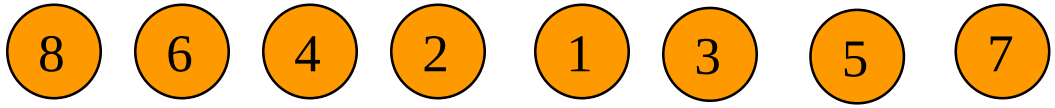
- currentTree = first subtree.
- for (each of the remaining trees)

```
currentTree = compareLink(currentTree,
                          nextTree);
```

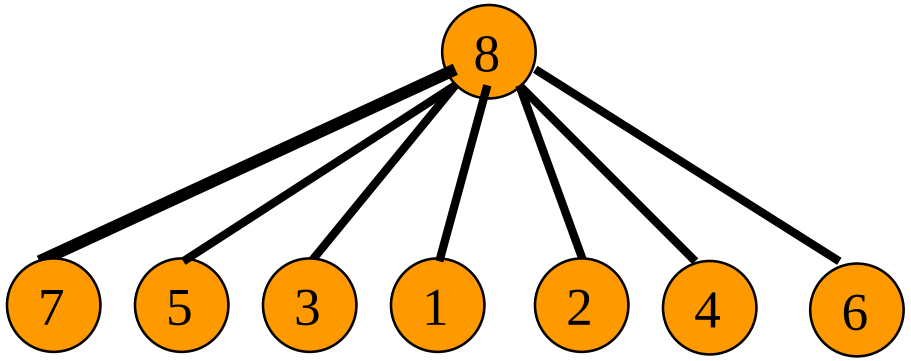
# Example




- Remove max.




- Meld into one tree.



# Example

- Actual cost of insert is 1.
- Actual cost of remove max is degree of root.
- n inserts (9, 7, 5, 3, 1, 2, 4, 6, 8) followed by n remove maxs.
  - Cost of inserts is n.
  - Cost of remove maxs is $1 + 2 + \ldots + n - 1 = \Theta(n^2)$.

# Good Ways To Meld Subtrees

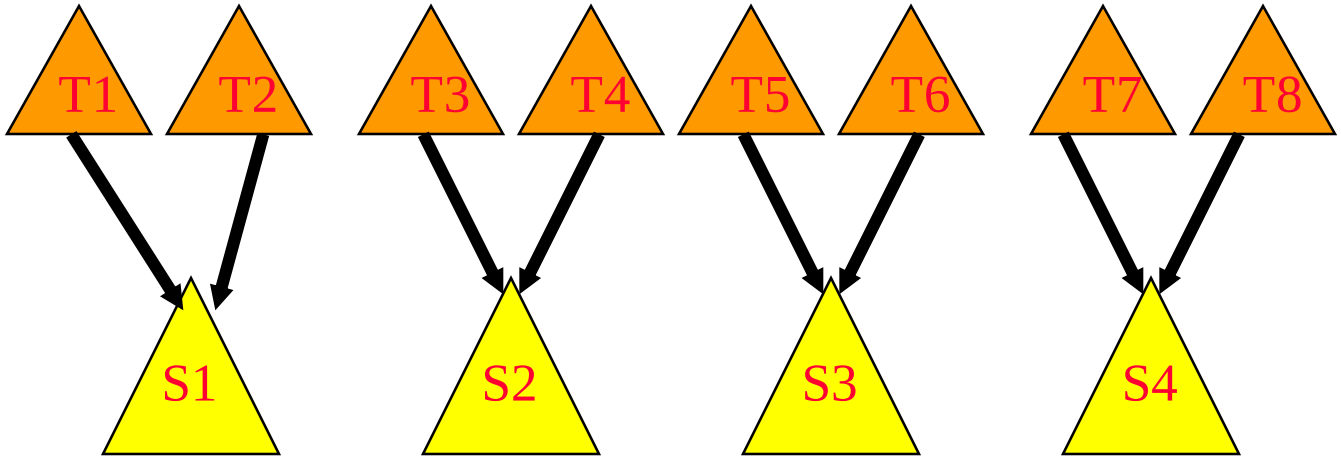
- Two-pass scheme.
- Multipass scheme.
- Both have same asymptotic complexity.
- Two-pass scheme gives better observed performance.

# Two-Pass Scheme

- Pass 1.
  - Examine subtrees from left to right.
  - Meld pairs of subtrees, reducing the number of subtrees to half the original number.
  - If # subtrees was odd, meld remaining original subtree with last newly generated subtree.
- Pass 2.
  - Start with rightmost subtree of Pass 1. Call this the working tree.
  - Meld remaining subtrees, one at a time, from right to left, into the working tree.

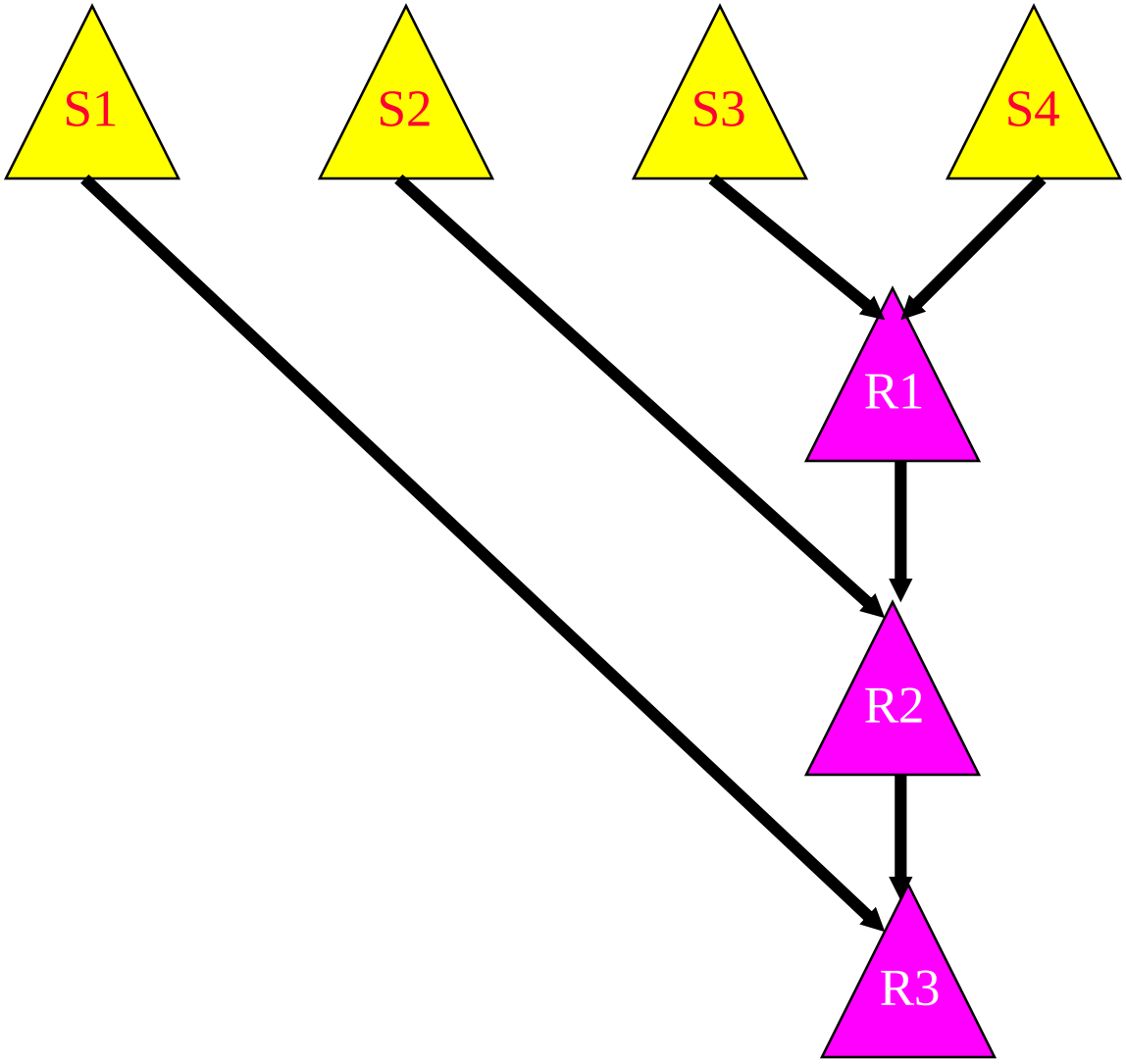
# Two-Pass Scheme – Example

**Pass 1**



# Two-Pass Scheme – Example

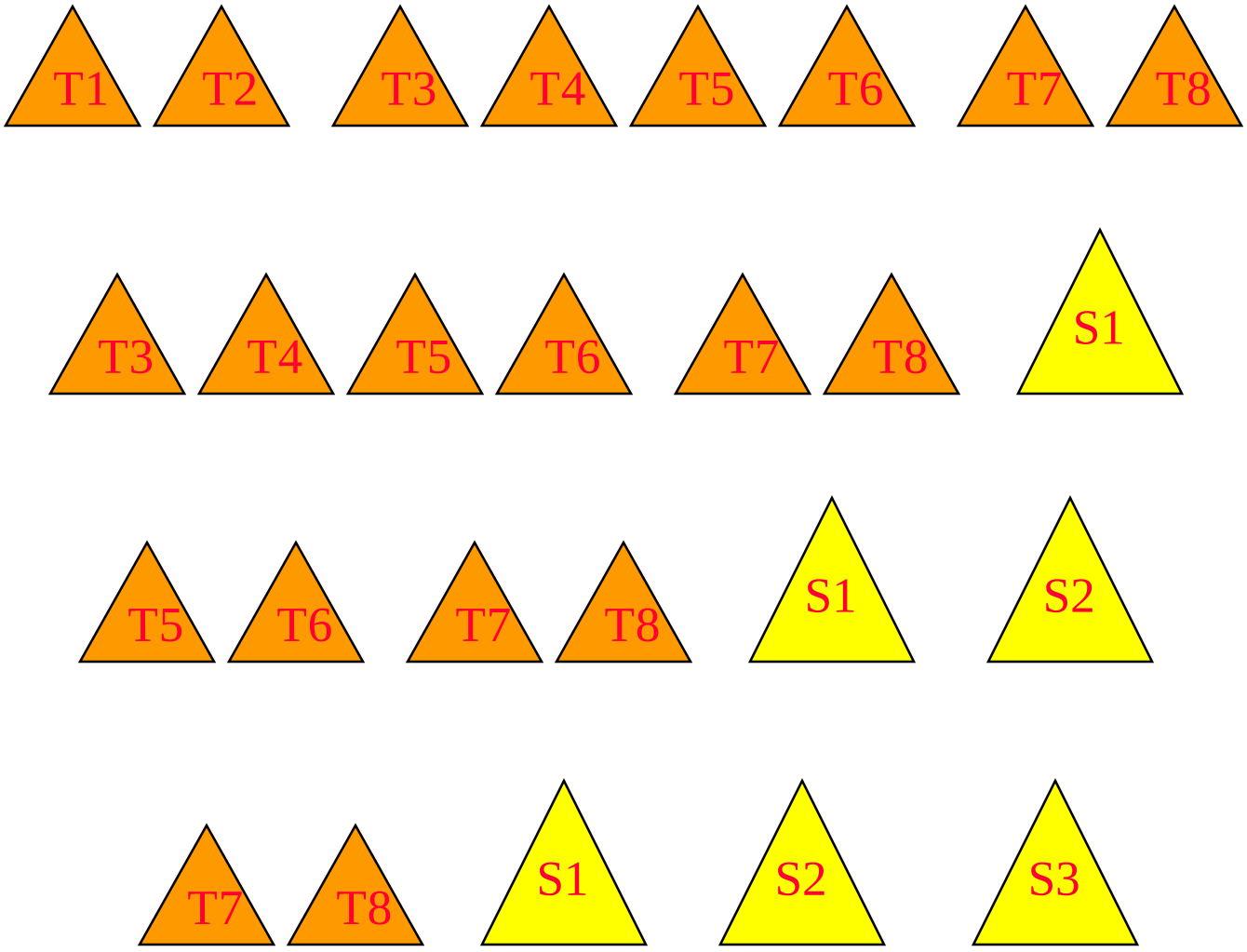
**Pass 2**



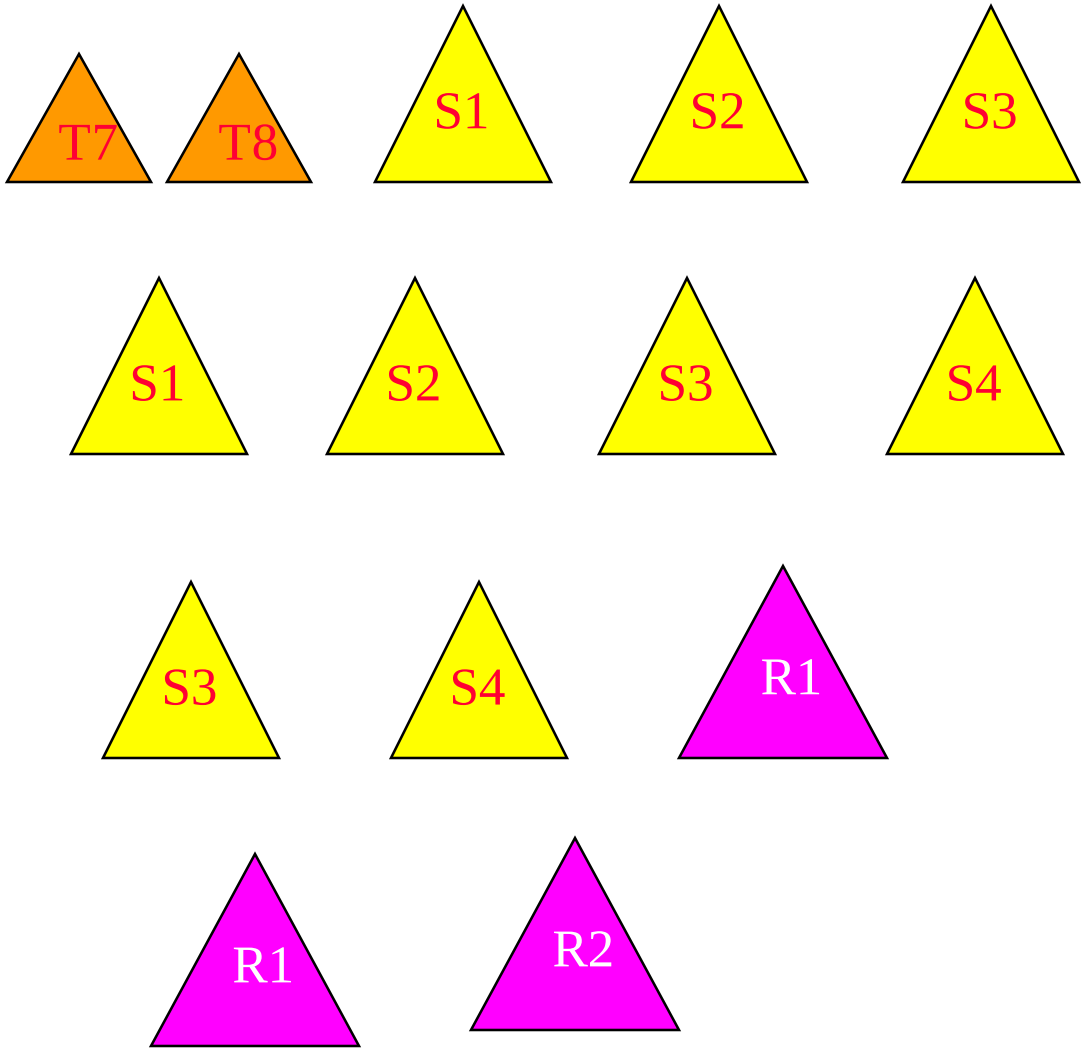
# Multipass Scheme

- Place the subtrees into a FIFO queue.
- Repeat until 1 tree remains.
  - Remove 2 subtrees from the queue.
  - Meld them.
  - Put the resulting tree onto the queue.

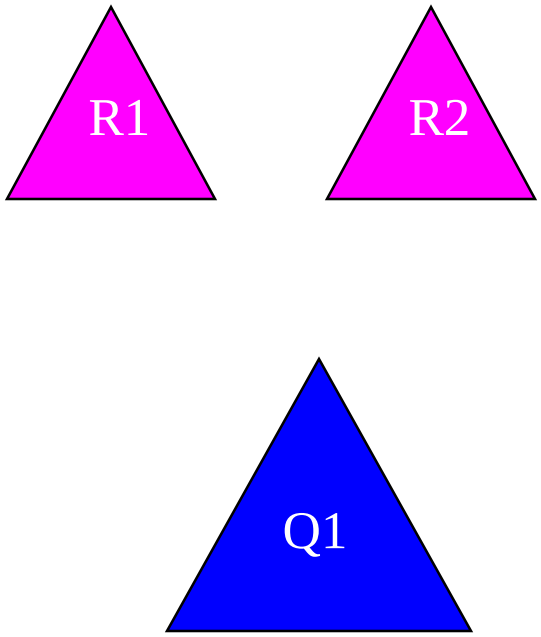
# Multipass Scheme – Example



# Multipass Scheme--Example



# Multipass Scheme--Example

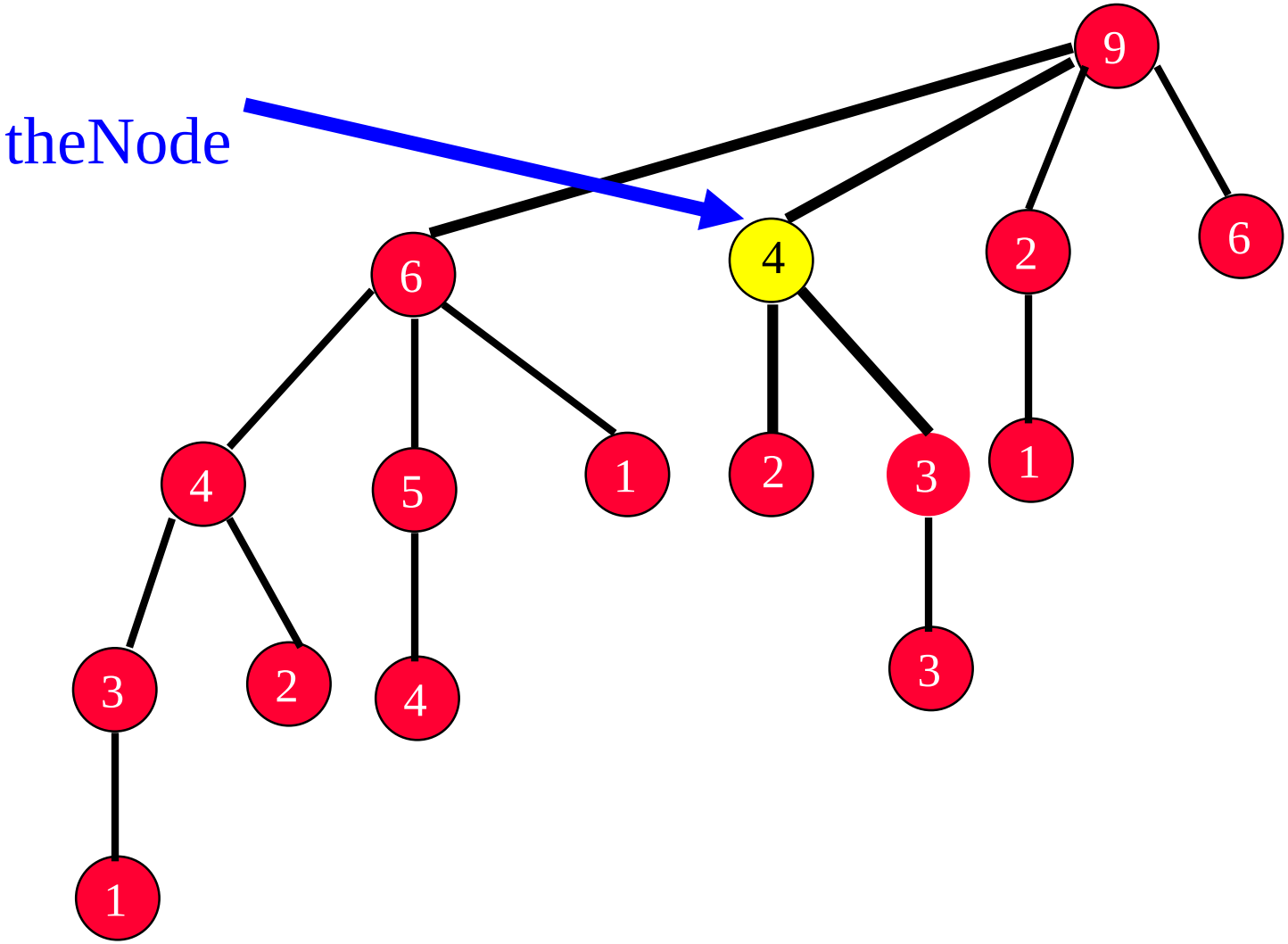



- Actual cost = O(n).

# Remove Nonroot Element

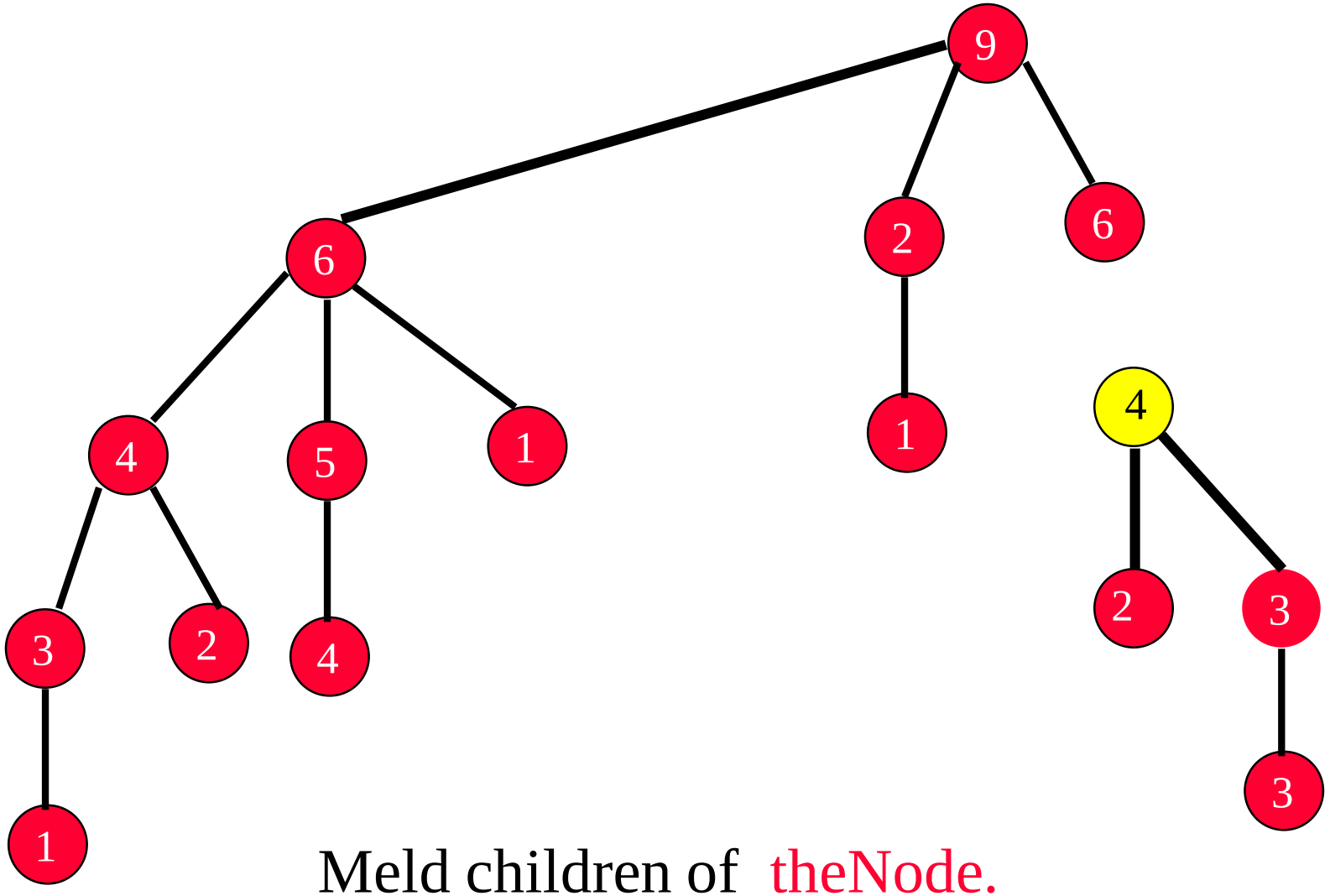
- Remove theNode from its sibling list.
- Meld children of theNode using either 2-pass or multipass scheme.
- Meld resulting tree with what's left of original tree.

# Remove(theNode)



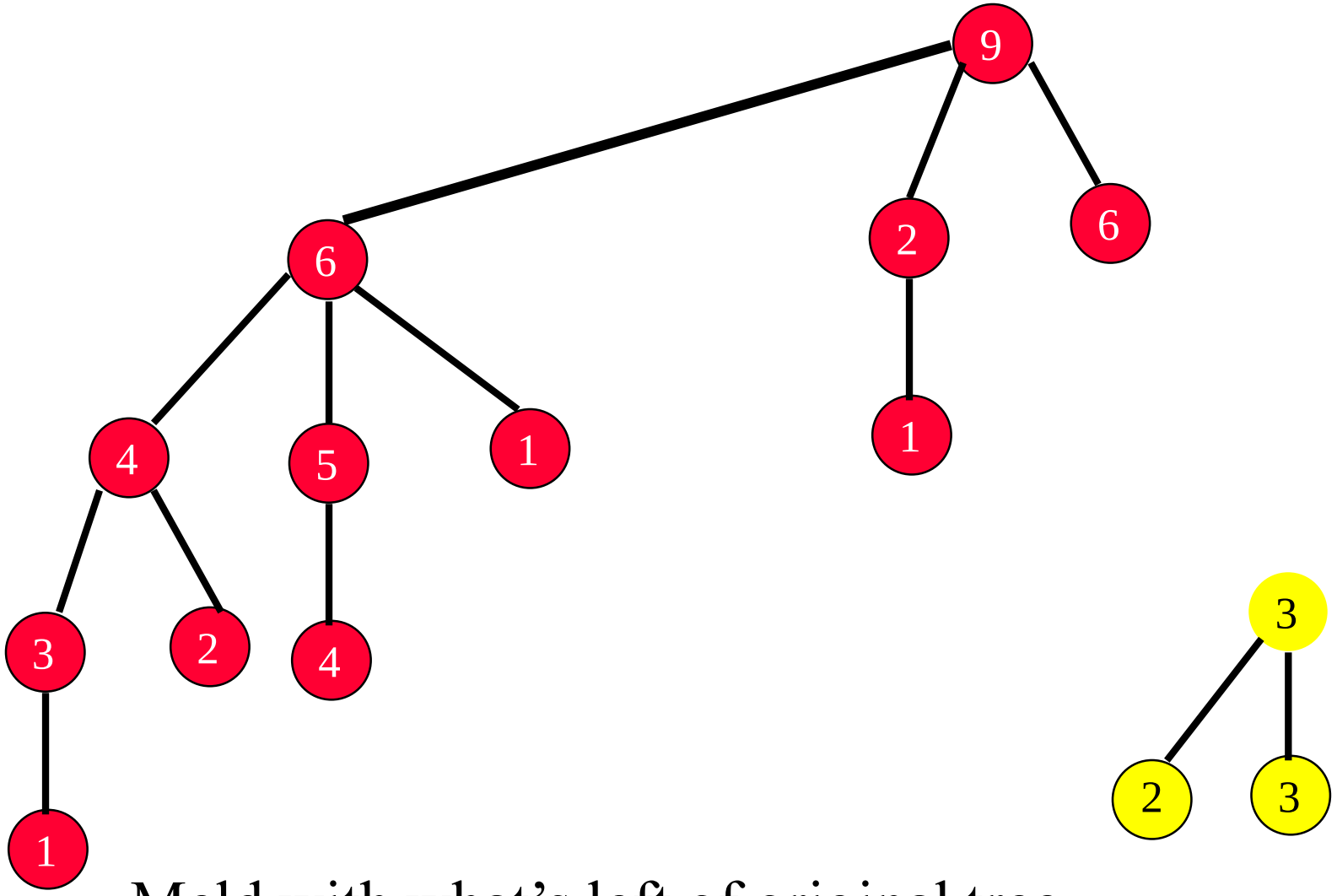

Remove theNode from its doubly-linked sibling list.

# Remove(theNode)



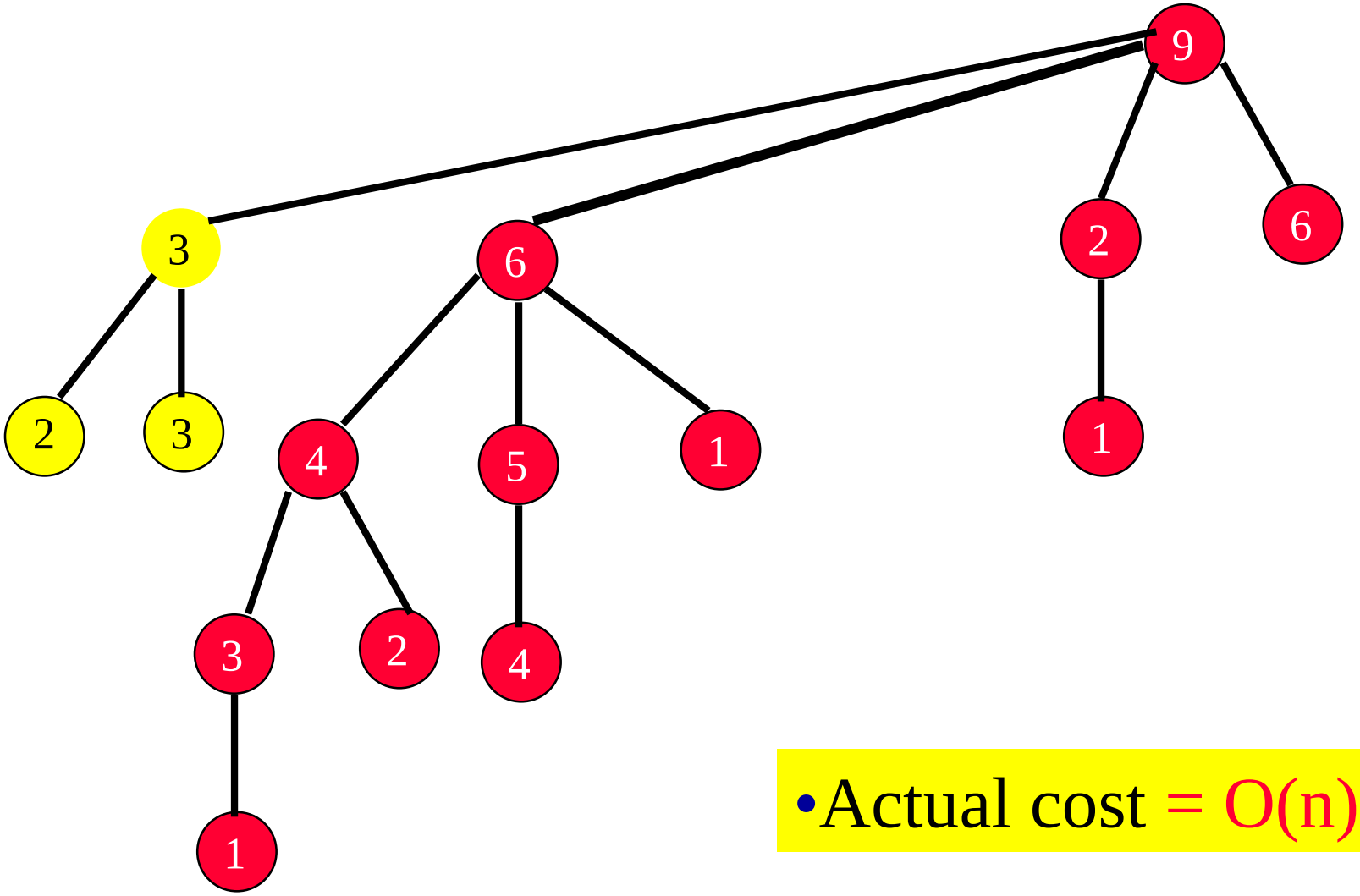

Meld children of theNode.

# Remove(theNode)




Meld with what's left of original tree.

# Remove(theNode)




- Actual cost = O(n).

# Acknowledgements


- E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of data structures in C++, Second Edition, Silicon Press, 2007.
- Lecture Slides of Prof Sartaj Sahni.